

Smart Split

bunq Hackathon 7.0 · Multimodal AI in Banking

The problem

Splitting a restaurant bill is not solved. Splitwise and Tikkie require someone to type every item manually -- so groups default to rounding and someone always subsidises someone else's meal. The actual blocker isn't payment (bunq handles that). It's the chaotic 5 minutes between the bill arriving and everyone knowing what they owe.

THE PRODUCT

How Smart Split Works

Smart Split lives inside bunq as a native feature on any outgoing payment. One person pays, opens the transaction, and the entire split happens from there — no separate app, no manual entry.

1

Open the transaction, tap Smart Split

The person who paid opens the restaurant charge in bunq. Alongside 'Add note' and 'Attach receipt' sits a new option: Smart Split. Tapping it launches the camera.

2

Photograph the receipt — a standardised bunq receipt is generated

The AI reads every line item, quantity, and price. Within 2–3 seconds the raw receipt becomes a clean, bunq-styled list of checkboxes — one per item. VAT and service charge are broken out separately.

Validation runs immediately: sum of all items + service charge + VAT must equal the bunq transaction amount. If there's a gap (e.g. an unlisted cover charge), it surfaces as an unassigned line rather than silently distorting everyone's share. The math closes before anything is sent.

3

Everyone picks what they had

The organiser invites people from their bunq contacts or generates a QR code the table scans. Each person taps the items they ordered on their own screen. Items update live across all devices. Shared items (a bottle of wine, a starter) split automatically when more than one person claims them. An 'Equal split' toggle is always one tap away for groups who don't want item-level precision.

4

One tap sends itemised payment requests

Each person receives a bunq Payment Request with a human-readable description — not just an amount, but the specific items they're paying for. Non-bunq users at the table receive a bunqme payment link via WhatsApp or SMS.

The organiser's transaction view gains a live tracker: avatars fill in green as people pay. The transaction stays 'partially settled' until all requests are accepted.

PRODUCT DETAIL

Key Decisions & Edge Cases

Shared items

Multiple people can tap the same item. It divides evenly among all claimants and turns a different colour to signal it's shared. No manual fractions needed.

Service charge & tip

If a service charge is on the receipt, it's distributed proportionally to each person's food total — not equally. An optional tip field appears at review if no charge was printed.

Rounding

Any rounding remainder goes to the organiser's share automatically, noted inline. They organised the bill — they absorb €0.02.

Editing

Prices and item names can be corrected with a single tap → inline text field. If the AI misreads something, the fix path is immediate and non-modal.

Early departures

The organiser can finalise and send one person's request before the rest of the group has finished assigning.

Non-bunq users

Anyone at the table without bunq joins via QR code on a lightweight web view and pays via a bunqme link — no app download required.

ENGINEERING

Technical Approach

bunq API integration

Transaction anchor	GET /v1/user/{id}/monetary-account/{id}/payment/{id} — the confirmed amount from bunq is the validation target for all receipt parsing.
Contacts	GET /v1/user/{id}/contact — populates the invite picker with existing bunq contacts.
Payment requests	POST /v1/user/{id}/monetary-account/{id}/request-inquiry — one request per person, description auto-populated with itemised breakdown.
Non-bunq payments	POST /v1/user/{id}/bunqme-tab — generates a Tikkie-equivalent link for non-bunq users.
Status polling	GET /v1/user/{id}/monetary-account/{id}/request-inquiry/{id} — drives the live settlement tracker.

Receipt parsing pipeline

Vision LLM (GPT-4o or Claude) with a strict structured-output prompt returning JSON. The schema is validated before the UI renders:

```
{
  "items": [
    {"name": "Margherita Pizza", "qty": 1, "unit_price": 14.50, "total": 14.50},
    {"name": "Heineken", "qty": 3, "unit_price": 4.50, "total": 13.50}
  ],
  "service_charge": {"amount": 5.60, "type": "fixed"},
  "vat": {"amount": 4.80, "rate": 9},
  "subtotal": 28.00,
  "total": 33.60
}
```

Post-processing checks: $\text{sum}(\text{item totals}) + \text{service_charge} + \text{VAT} \approx \text{transaction.amount}$. Discrepancies surface as a named unassigned line rather than silently failing.

Stack

Frontend	React web app — camera capture via getUserMedia, works on every phone without an app install.
Backend	FastAPI (Python) — receipt parsing, session management, bunq API calls.
Real-time sync	Supabase Realtime or Firebase — live item-assignment state shared across all participants.
Hosting	Railway or Fly.io — deploys in minutes for a hackathon environment.

EXECUTION

24-Hour Build Plan

Hours	Focus	Deliverable
0 – 2	Setup	bunq sandbox accounts live, repo initialised, hosting deployed, JSON schema agreed.
2 – 6	Receipt parsing	Vision pipeline tested on 10+ real receipts including Dutch ones. Validation logic passing
6 – 11	Core UI	Capture flow, item checklist, assignment interface, equal-split toggle, shared-item detect
11–16	bunq integration	QR join flow, contact selector, payment request firing, bunqme link for non-bunq users.
16–20	Polish & edge cases	Service charge logic, tip field, item editing, loading states, error messages.
20–23	Demo rehearsal	Full flow run 5+ times. Fallback video recorded. Pitch narrative finalised.
23–24	Buffer	Critical fixes only.

Team split (3 people)

Person	Owns
Frontend / UX	Capture flow, item checklist UI, real-time sync display, live tracker screen.
AI / Backend	Receipt parsing pipeline, validation logic, session management API, shared-item logic.
bunq Integration	API auth, payment request & contact endpoints, QR join flow, presentation slides, rehearsal

PRESENTATION

Demo Strategy

The demo is the pitch. Plan it like a product moment, not a code walkthrough.

0 - 15 s	State the problem Hold up a restaurant receipt. Ask the audience: "who's done this at a table?" Pause. Then: "This is what we fixed."
15 - 45 s	Live receipt scan Photograph the prop receipt on stage. Items appear on screen in ~2 seconds. Let the audience read them. Don't narrate -- let the moment land.
45 - 75 s	Live item assignment A second phone on stage (or a volunteer from the audience) scans the QR and joins. Assign items across both devices. The audience sees real-time sync.
75 - 90 s	Send & settle Tap Send. Show a bunq payment request arrive on the second phone. Optionally pre-record a 10-second clip of the tracker turning green as a safety net.

JUDGING

Why This Scores Well

Criterion	Weight	Why Smart Split scores
Impact & Usefulness	30%	Universal recurring pain — anyone who eats out in a group is the target market
Innovation	25%	Not a new concept, but the execution is: anchored to the bunq transaction, not a generic receipt
Technical Execution	20%	Receipt parsing with structured validation, real-time sync, and full bunq API integration
bunq Integration	15%	Payment requests, contacts, bunqme tabs, and transaction anchoring are all implemented

Criterion	Weight	Why Smart Split scores
Presentation	10%	The demo is visual, live, and involves the audience. The "wow" moment (re